Advanced Server-Side Web Programming

6COSC019C.2

Report

Author: Jason Mishike Lathenial Carvalho

Student ID: w1901978

Table of Contents

Course Work 1 2
Overall design 3
MVC architecture 3
Load Balancing Diagram 5
Routing diagram 6
Entity Relationship Diagram 7
Technology selection 8
Big Picture Choices 9
Database related choices 9
Security related choices 9
Email related choices 10
Database Design 10
Database Normalization 11
API Design 11
RESTful API 11
Objectives achieved 11
Course Work 2 12
Overall design 13
High Level MVC 13
Detailed MVC Diagram 13
Load Balancing diagram 14
Routing diagram 14
CORS protection 15
API security (Cookies, CSRF, API keys) 16
Rate limiting diagram 17
Technological choices 18
Objectives Achieved 19
Appendix 19
API Specifics 20

Authentication 20
Profile 20
Sponsorship 21
Bidding 22
Public API 22
API Client Management 22
Admin 23
UAT testing 23

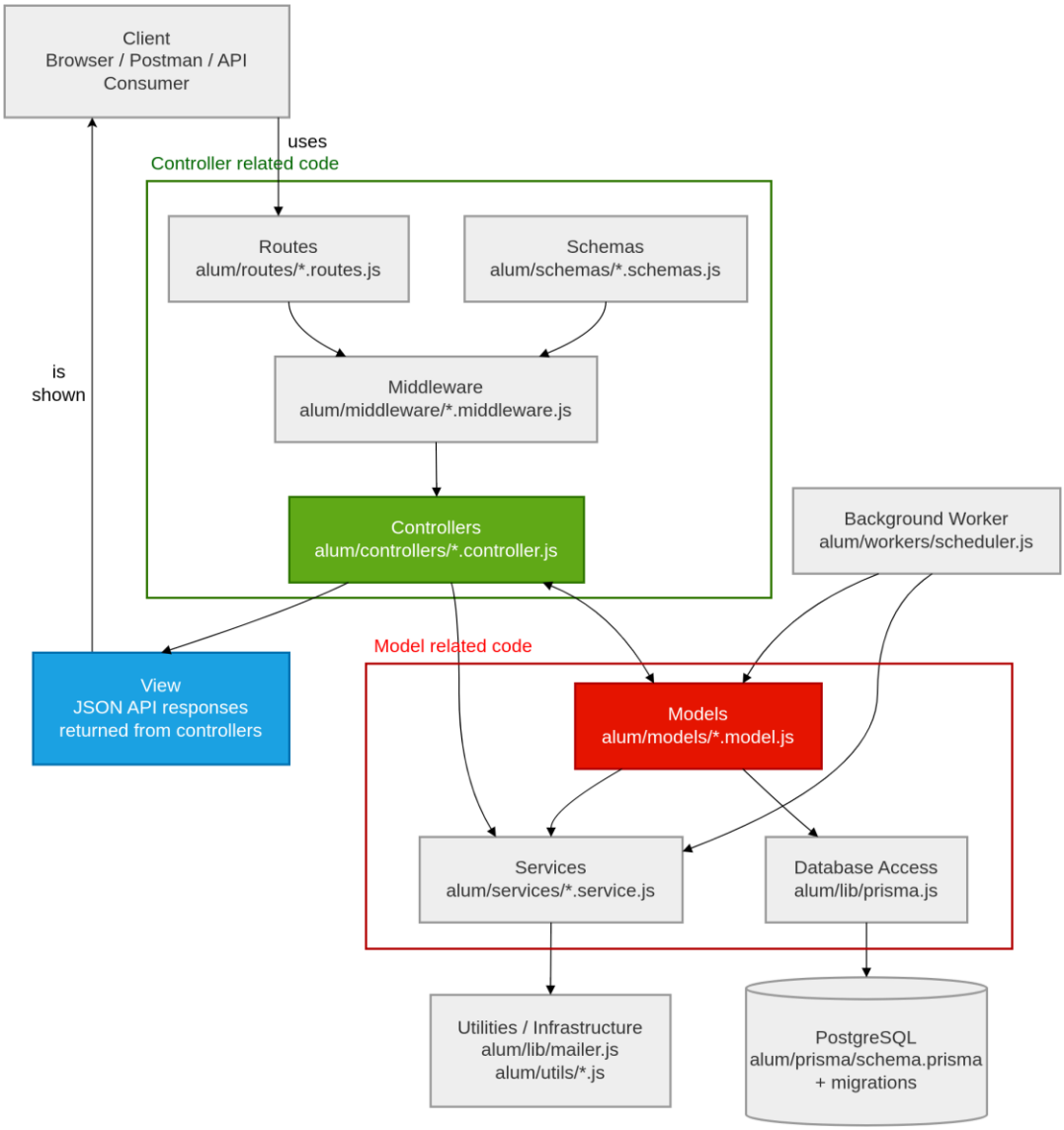
Course Work 1

Overall design

MVC architecture

The system was built with an MVC architecture in mind, the only differences from the traditional approach is that the views are not seperate pages/elements but rather the JSON responses and since we are using ExpressJS for implementation we mainly use functions grouped in modules in line with the ExpressJS's middleware approach over using strictly defined classes.

High level MVC diagram



The Model, View and Controller classes are separated in different directories in order to make the separation easy to identify and maintain.

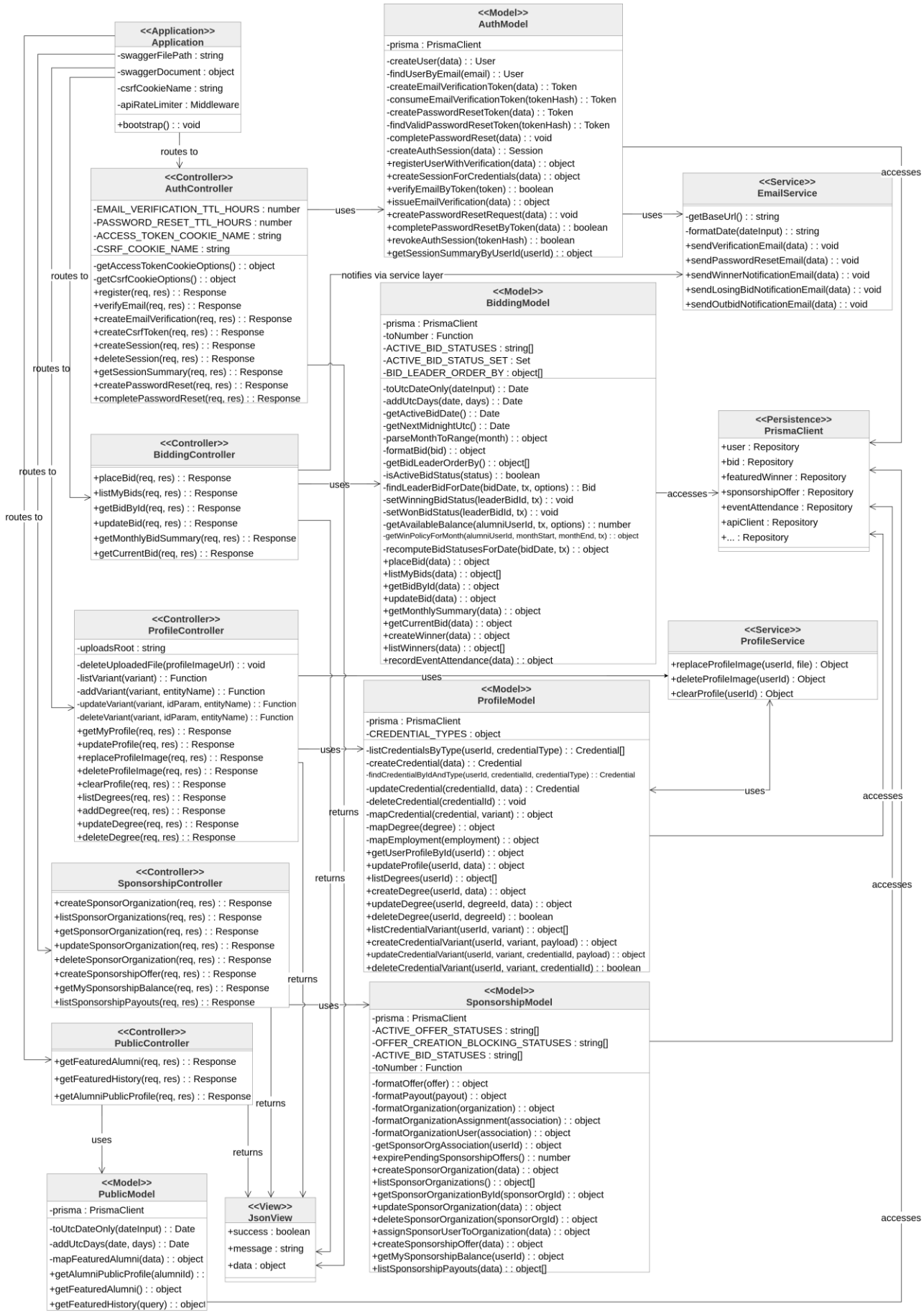
Detailed MVC Diagram

The below diagram (in next page as it is large) has the modules and functions shown as classes with each of the attributes and methods shown appropriately. Please note prisma was used as the ORM and so is used in each of the models to store data to the database.

Furthermore, in line with best practices, attributes are not exposed directly and only through the functions.

JsonView is not an actual class, but a representation of what the response object the client would receive is.

Jason Carvalho | Advanced Server Side

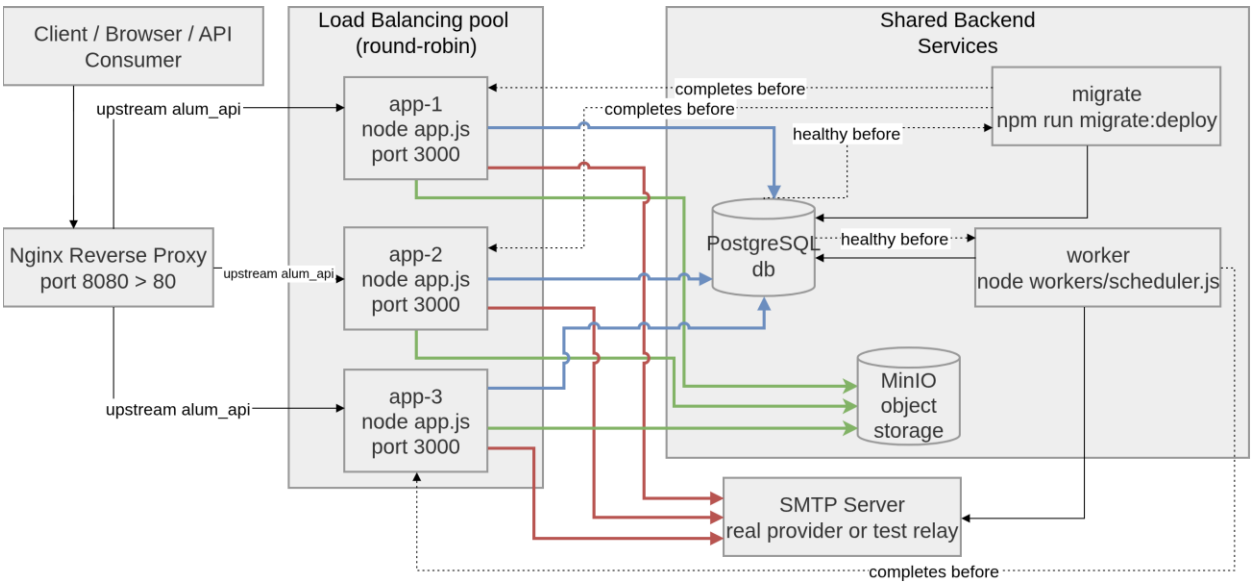


Load Balancing Diagram

The ExpressJS application is made with horizontal scaling in mind. Requests are stateless in line with RESTFul API standards. Data is stored on shared backend, composing of PostgreSQL for the database and MinIO as shared file storage.

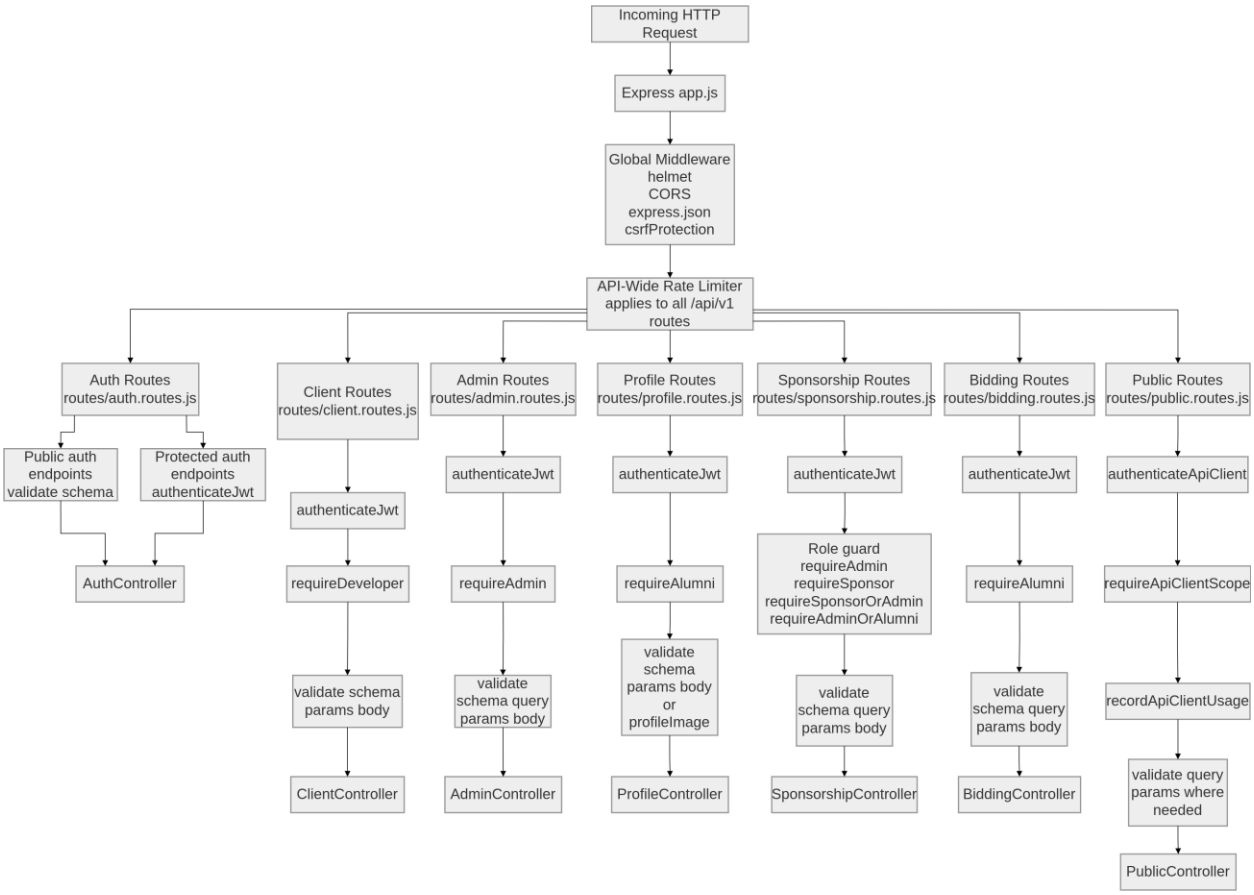
In the docker-compose.prod, database migration happens as a separate service and not on app start up to prevent conflicts if different instances of the app run migrate. Similarly, worker which chooses winner at midnight and handles expiring sponsorship offers run in a separate container and can be scaled independently.

The round-robin load balancing is handled by a nginx container. The technology choice is defended in the technology selection section below.



Routing diagram

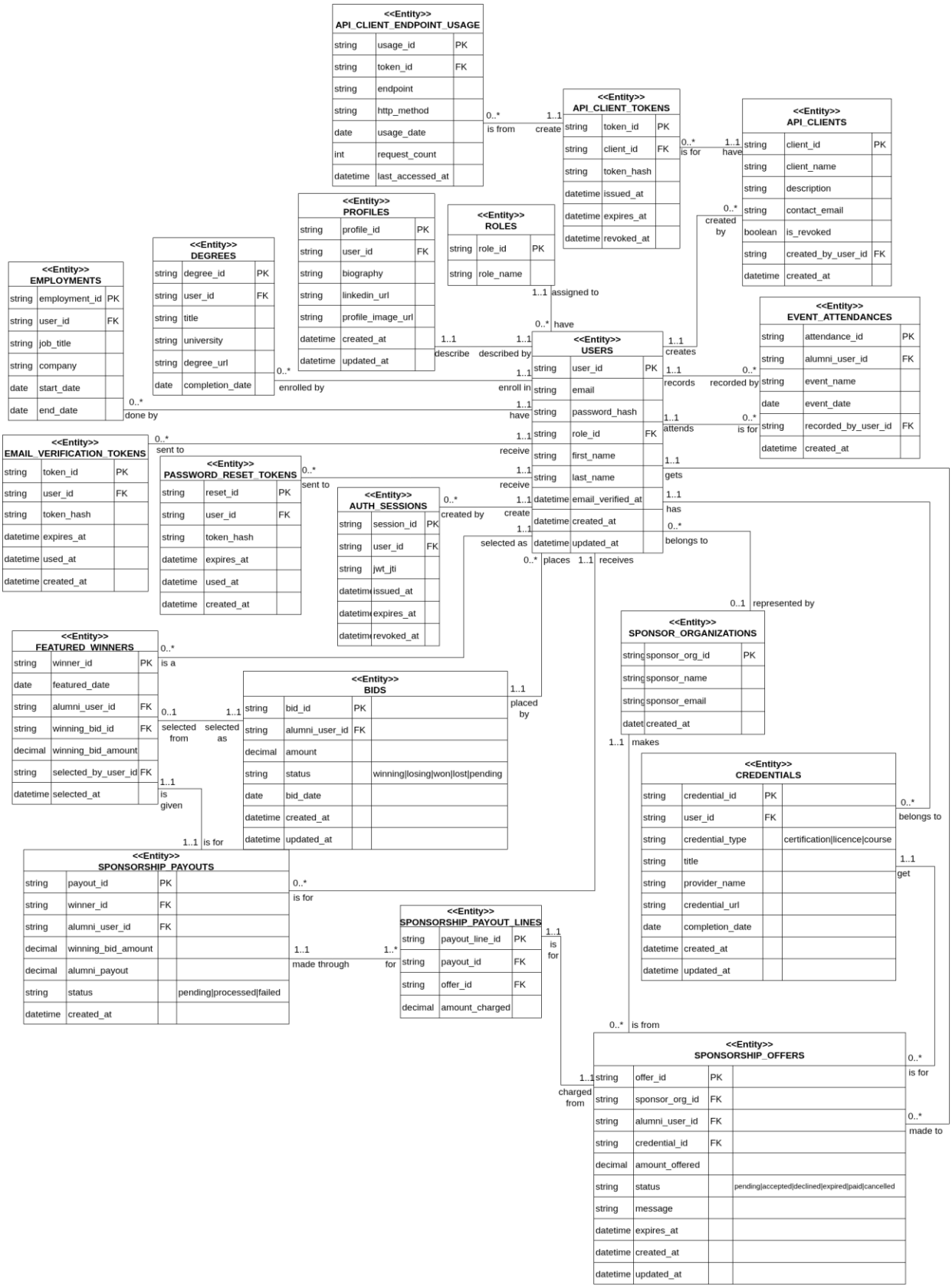
The routing diagram shows the overall lifecyle of a request and also the general structure of the API.



Entity Relationship Diagram

You can find the high-quality diagram at: https://raw.githubusercontent.com/jasonmishi/ASS-CW1/refs/heads/main/alum/docs/ER%20diagram.drawio.png

3NF is done to reduce data redundancy and make transactions simpler.


<<Entity>> API_CLIENT_ENDPOINT_USAGE
string usage_id PK
string token_id FK
string endpoint
string http_method
date usage_date
int request_count
datetime last_accessed_at
0..* is from
1..1 create
<<Entity>> API_CLIENT_TOKENS
string token_id PK
string client_id FK
string token_hash
datetime issued_at
datetime expires_at
datetime revoked_at
0..* is for
1..1 have
<<Entity>> API_CLIENTS
string client_id PK
string client_name
string description
string contact_email
boolean is_revoked
string created_by_user_id FK
datetime created_at
0..* created by
<<Entity>> PROFILES
string profile_id PK
string user_id FK
string biography
string linkedin_url
string profile_image_url
datetime created_at
datetime updated_at
<<Entity>> ROLES
string role_id PK
string role_name
1..1 assigned to
0..* have
<<Entity>> DEGREES
string degree_id PK
string user_id FK
string title
string university
string degree_url
date completion_date
0..* enrolled by
<<Entity>> EMPLOYMENTS
string employment_id PK
string user_id FK
string job_title
string company
date start_date
date end_date
0..* done by
1..1 describe
1..1 described by
<<Entity>> USERS
string user_id PK
string email
string password_hash
string role_id FK
string first_name
string last_name
datetime email_verified_at
datetime created_at
datetime updated_at
1..1 enroll in
1..1 have
1..1 creates
1..1 records
0..* recorded by
1..1 attends
0..* is for
<<Entity>> EVENT_ATTENDANCES
string attendance_id PK
string alumni_user_id FK
string event_name
date event_date
string recorded_by_user_id FK
datetime created_at
0..* sent to
1..1 receive
<<Entity>> EMAIL_VERIFICATION_TOKENS
string token_id PK
string user_id FK
string token_hash
datetime expires_at
datetime used_at
datetime created_at
0..* sent to
1..1 receive
<<Entity>> PASSWORD_RESET_TOKENS
string reset_id PK
string user_id FK
string token_hash
datetime expires_at
datetime used_at
datetime created_at
0..* created by
1..1 create
<<Entity>> AUTH_SESSIONS
string session_id PK
string user_id FK
string jwt_jti
datetime issued_at
datetime expires_at
datetime revoked_at
1..1 gets
1..1 has
0..* belongs to
selected as
0..* places
1..1 receives
0..1 represented by
<<Entity>> SPONSOR_ORGANIZATIONS
string sponsor_org_id PK
string sponsor_name
string sponsor_email
datetime created_at
1..1 makes
<<Entity>> FEATURED_WINNERS
string winner_id PK
date featured_date
string alumni_user_id FK
string winning_bid_id FK
decimal winning_bid_amount
string selected_by_user_id FK
datetime selected_at
0..* is a
0..1 selected from
1..1 selected as
1..1 is given
1..1 is for
<<Entity>> BIDS
string bid_id PK
string alumni_user_id FK
decimal amount
string status winning|losing|won|lost|pending
date bid_date
datetime created_at
datetime updated_at
1..1 placed by
<<Entity>> CREDENTIALS
string credential_id PK
string user_id FK
string credential_type certification|licence|course
string title
string provider_name
string credential_url
date completion_date
datetime created_at
datetime updated_at
0..* belongs to
1..1 get
<<Entity>> SPONSORSHIP_PAYOUTS
string payout_id PK
string winner_id FK
string alumni_user_id FK
decimal winning_bid_amount
decimal alumni_payout
string status pending|processed|failed
datetime created_at
0..* is for
1..1 made through
1..* for
<<Entity>> SPONSORSHIP_PAYOUT_LINES
string payout_line_id PK
string payout_id FK
string offer_id FK
decimal amount_charged
1..1 is for
0..* is from
1..1 charged from
0..* is for
0..* made to
<<Entity>> SPONSORSHIP_OFFERS
string offer_id PK
string sponsor_org_id FK
string alumni_user_id FK
string credential_id FK
decimal amount_offered
string status pending|accepted|declined|expired|paid|cancelled
string message
datetime expires_at
datetime created_at
datetime updated_at

Technology selection

ExpressJS was selected for implementation from the choices presented for the coursework.

Big Picture Choices

Containerization: Docker and docker compose were used for containerization and orchestration for easy reproducibility and deployment.

Model-View-Controller (MVC) architecture: The MVC architecture pattern was applied strictly through the application (even at the cost of readability in some cases). Separations enforced through file structure.

Schema/Specification driven development: A thorough OpenAPI (swagger) specification was created at the start of the project and refined through implementation and testing. The schema serves as the source of truth for which the application is built. Other documentation is also provided to increase transparency and understanding. Swagger doc is hosted at /api-docs

Test Driven Development: Emphasis on automated testing is put in the development process. Both unit testing and integration testing is handled via jest (in the case of integration test additionally docker is used). E2E testing is done via Postman (the collection is contained in the repository). A test diamond (https://web.dev/articles/ta-strategies#testing_trophy) style strategy is followed, with static testing being handled by SonarQube (https://sonarcloud.io/project/overview?id=jasonmishi_ASS-CW1)

Horizontal scaling: Was kept in mind while designing the system to ensure that the code is suitable for prod. Important data is stored and retrieved from the database and the API is kept stateless. Timing based database updates (winner selection, offer expiry) are carried out via a worker that can be deployed separately, ensuring no runtime or database write conflicts due to duplicate transactions. Nginx was used as a load balancer in production docker compose file to do round-robin load balancing. Minio is used as shared file storage to enable shared access across horizontally scaled instances.
The rate-limiting is also database driven to ensure statelessness.

Database related choices

Postgres was picked due to existing developer familiarity. Compared to other production equivalent relational databases such as MySQL, it has very strong standards of compliance for SQL. Furthermore, it supports more advanced features and is a widely supported and battle tested FOSS product.

Prisma was selected because of the balance it gives between performance, flexibility, and increasing developer productivity. Prisma also has first class support for PostgreSQL. It gives easy access to features that increase the reliability of systems such as database migrations. The main cost is typically worse performance in applications that need complex SQL; however, this application does not need complex SQL so is not affected.

Security related choices

Cross Origin Resource Sharing (CORS): Used to prevent malicious websites making requests to this website through the browser by restricting the domains allowed to call the website. Implemented through custom middleware using the cors library.

Cross-Site Request Forgery (CSRF): Used to prevent the server from responding to malicious requests by requiring a unique token to be passed for mutable requests. CSFR tokens are implemented through custom middleware where tokens are generated through node:crypto. The header used to check the CSFR tokens in each request is X-CSRF-Token. The Double-submit cookie pattern is implemented, due to it being more stateless and can be implemented on requests that don't need a session.

Cookie based JSON Web Token (JWT): When users successfully are authenticated, short-lived tokens are granted to the client to authenticate requests. These are in the form of securely generated JWT tokens. These are set as cookies. It could have been set as a value in the Header, but then CSRF would not have been necessary.

SQL injection protection: No raw SQL is run, and all database transactions via Prisma which protects against SQL injection, where the user can execute malicious SQL statements via inputs.

Bearer API and scoping: API keys are generated and handled for the client/ endpoints where tokens need to be passed in the Authorization header. Scoping features for these endpoints is also present to follow the security principle of least access.

Security Headers: HelmetJS used to add security headers to responses.

Email verification: Alumni email verification from domain is required before being able to log-in and use the system.

Request structure validation: request body structure is checked and validated using zod schemas for each request body required by the endpoints.

Rate limiting: express-rate-limit was used to apply rate limiting on all endpoint through middleware in order to prevent malicious users from spamming endpoints to get certain details or attempt certain attack vectors such as brute forces.

Email related choices

Mailhog, nodemailer: Mailhog is used for SMTP based testing locally, to make sure that the email related functionality is working, it is run through the docker-compose file, and emails are sent using the nodemailer library.

Database Design

Database Normalization

Database was normalized to 3rd Normal Form.

API Design

RESTful API

The RESTful architecture is defined by:

- HTTP compliant, where the API follows the http specifications and recommendations.
- Uniform interface where data is transferred in a standard format, where resources are identified uniquely with information on how the resources can be used.
- Requests are stateless so that the server can respond to every request independently

RESTful API architecture was followed to make the API in line with best practices and make it reliable.

Please check the OpenAPI schema for request body and response structure details. You can see it rendered in swagger at the route /api-docs.

The API endpoints are detailed further in the appendix

Objectives achieved

A secure, horizontal scalable, advanced server-side API application was created using ExpressJS framework. The database is designed and normalized to 3NF. The application has implemented security best practices with passwords being properly encrypted, CSFR, CORS and XSS protections. Rate-limiting to sensitive endpoints and secure API token creation and revocation with scoping for the tokens.

Documentation is provided and hosted in the application including API documentation, documentation for the env variables and instructions to get the application up and running both for development and production set-up. Code is kept simple and self-explanatory with comments where necessary.

Testing was created and documented, including the unit test that also help document the code, integration tests that make sure logic works against the database, and smoke tests to be used for deployments. Additionally, UAT tests are performed and documented in the Appendix of this document. A Postman collection was created and is present in the repository.

Business logic is properly translated from the requirements to application code. Including the account creation and management, alumni profile management, blind bidding for alumni of the day, sponsorship system, client management and client API token access system.

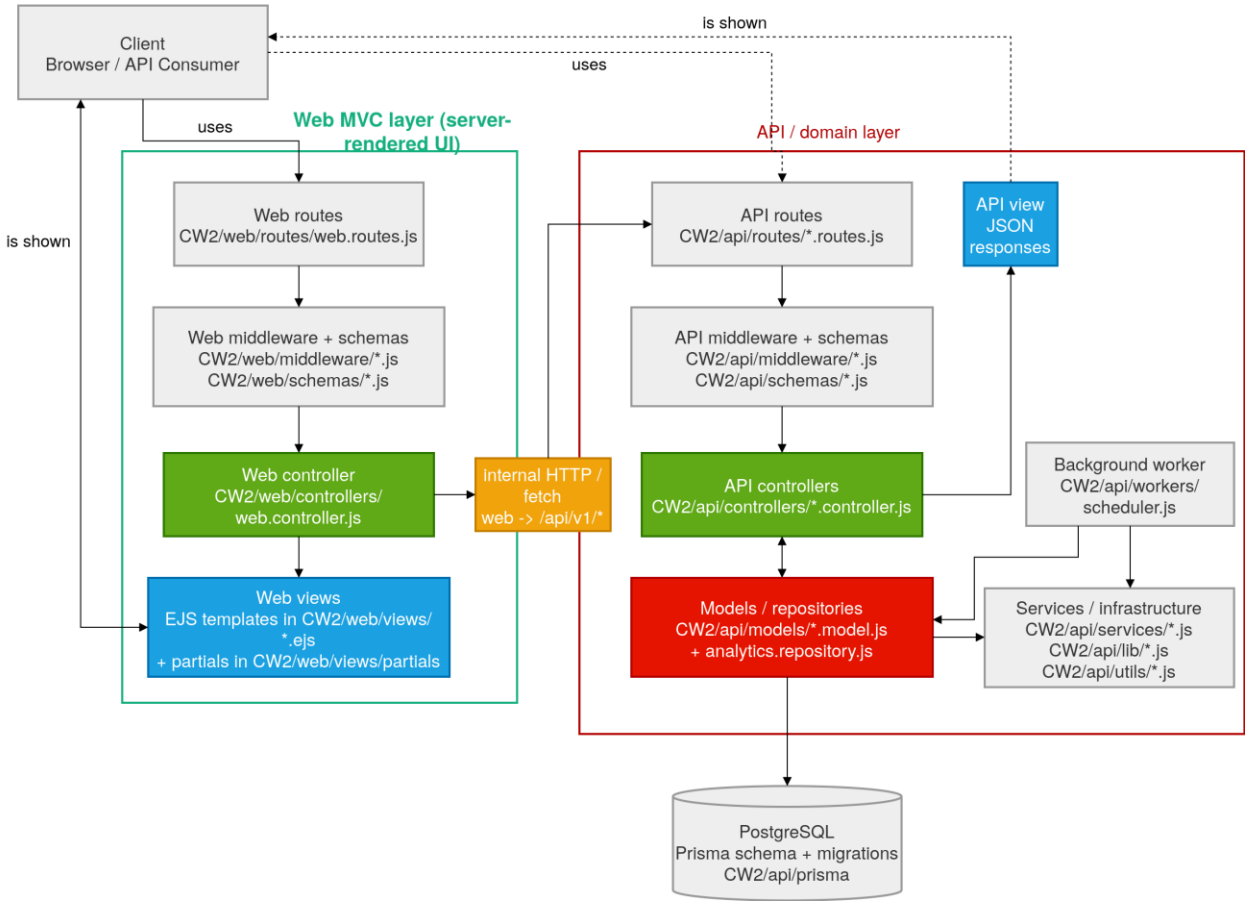
Architecture designs are documented in this report and choices have been defended.

Course Work 2

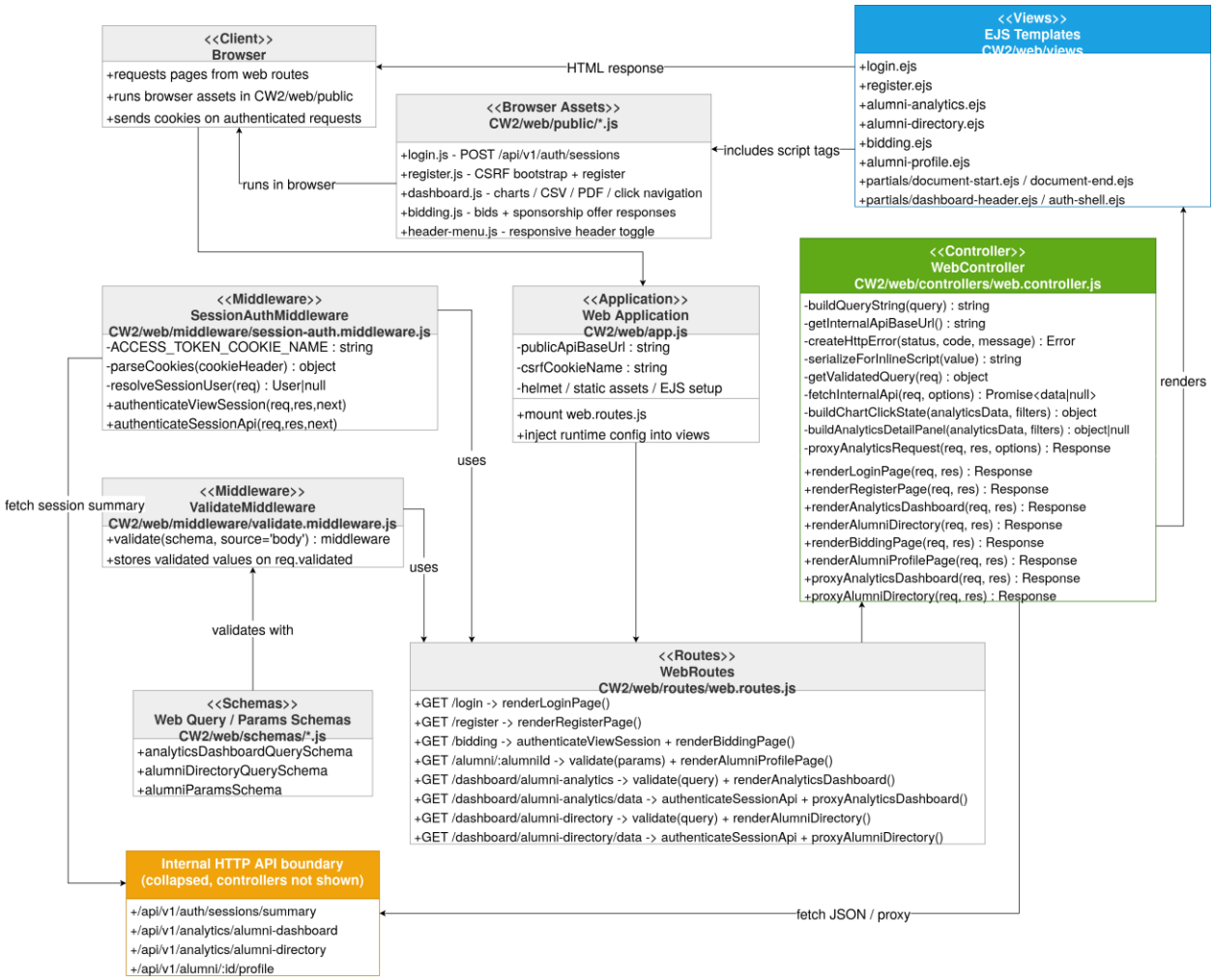
Overall design

High Level MVC

The frontend application implements the views and controllers, while the models stored in the api application is accessed through a server-side API request. The same security controls such as access tokens are proxied through with this request.

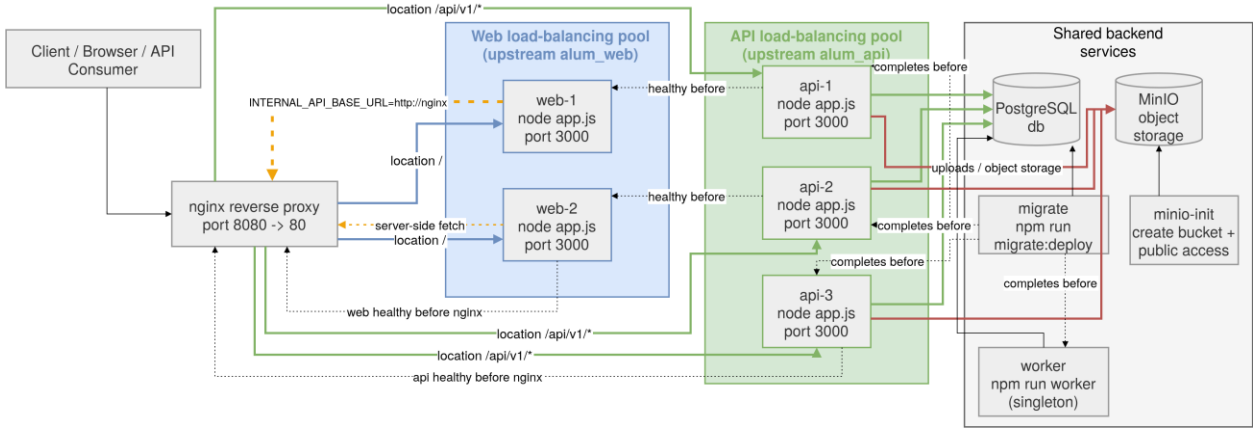


Detailed MVC Diagram

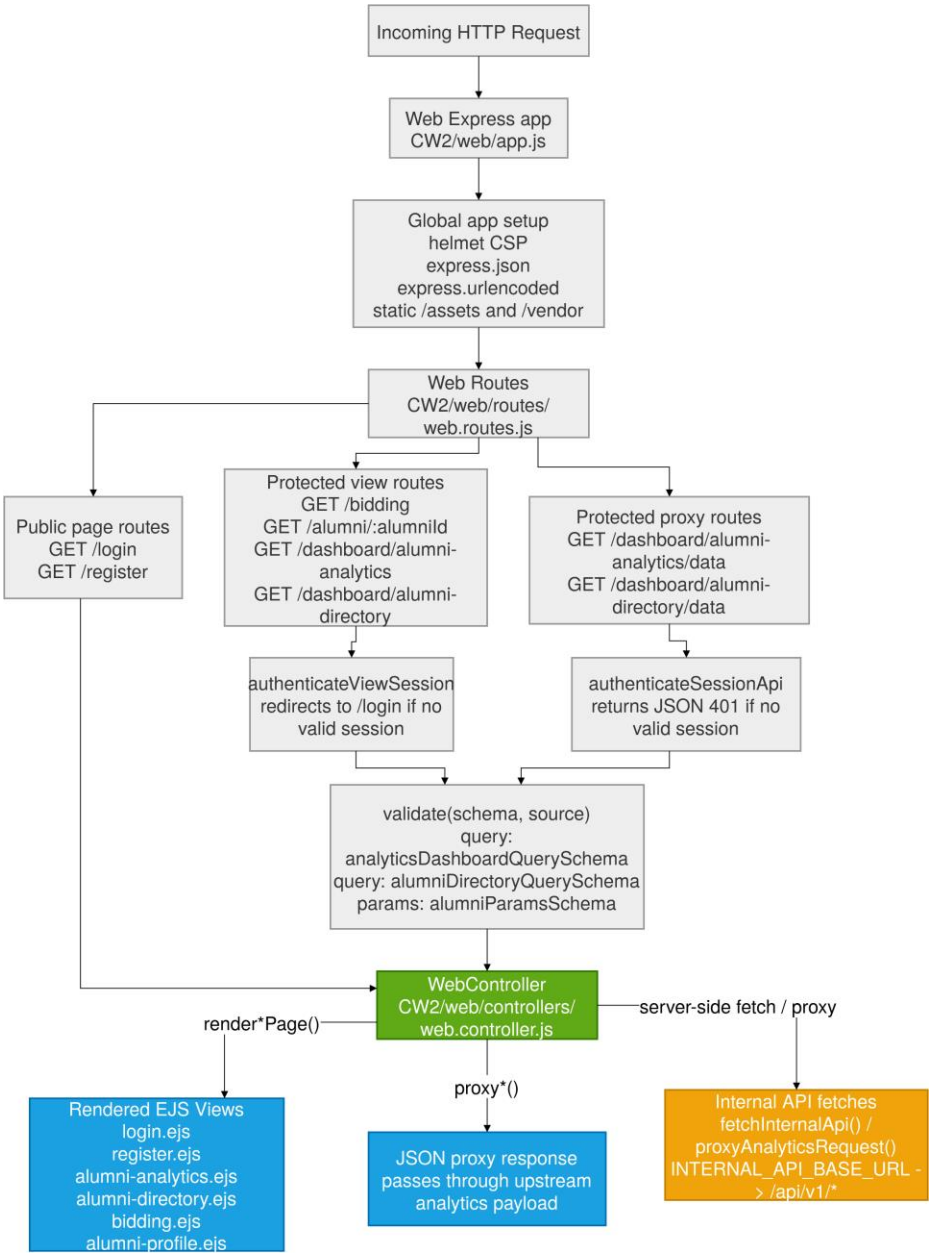


Load Balancing diagram

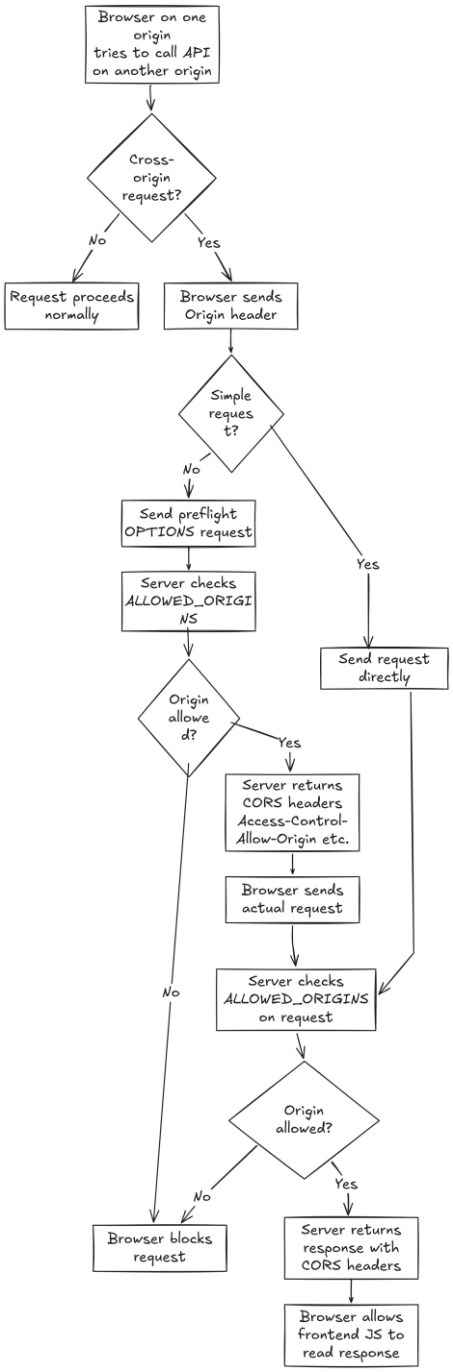
Same as before but now web containers are also added to be load ballanced by nginx.



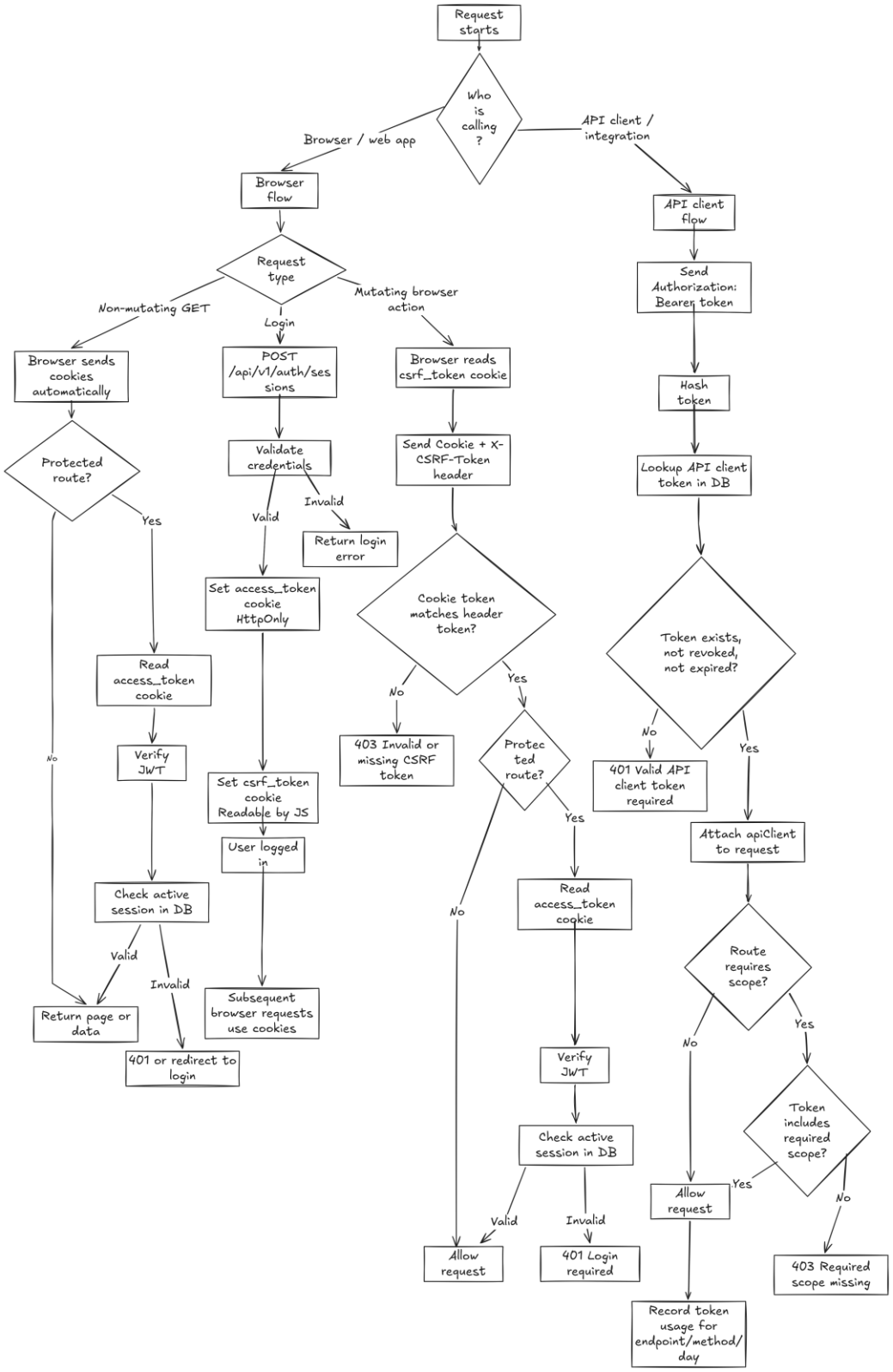
Routing diagram



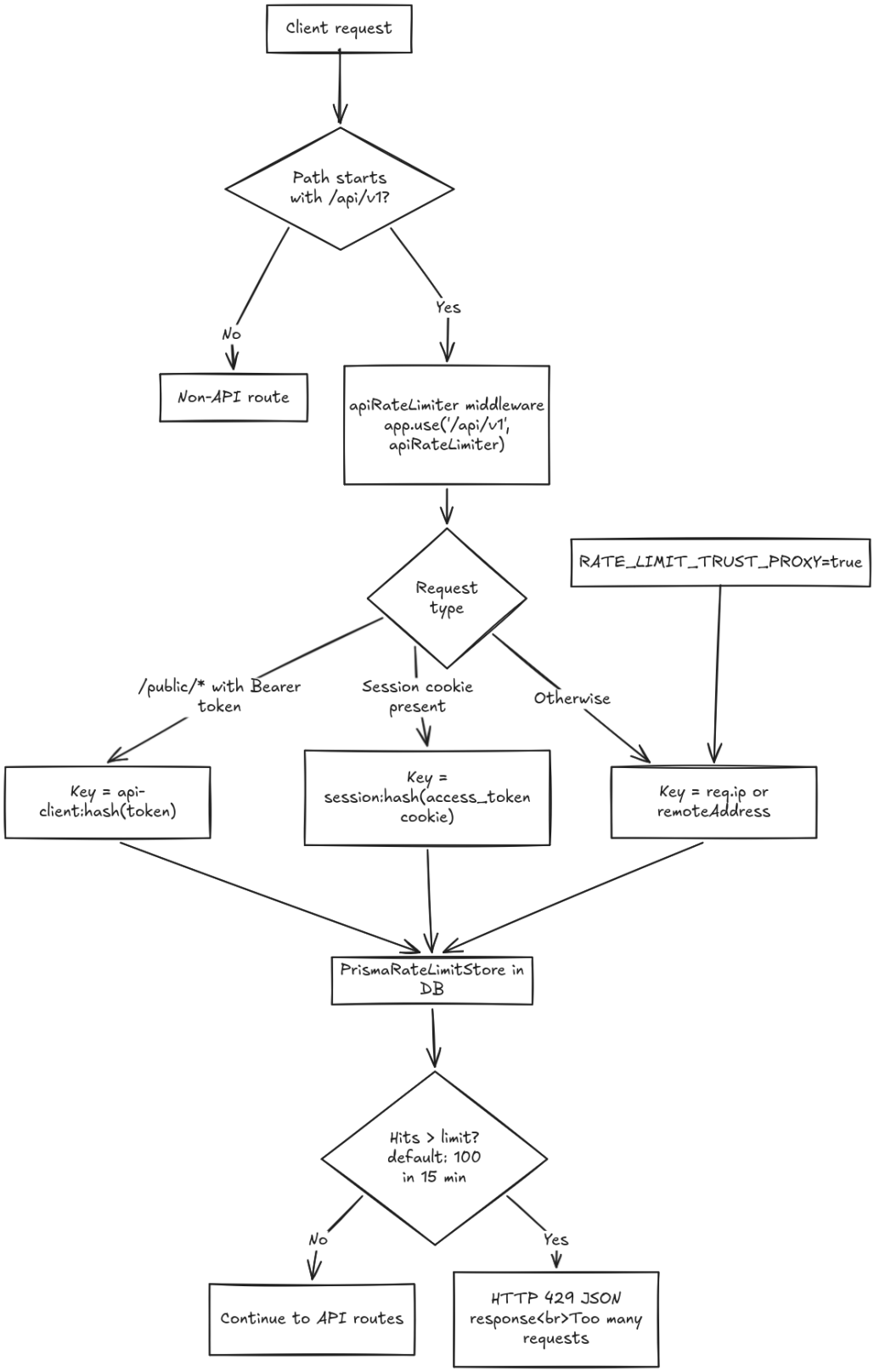
CORS protection



API security (Cookies, CSRF, API keys)



Rate limiting diagram



Technological choices

This is an extension of the technological choices for CW1, so the same justifications carry forward, what is listed below are the additional CW2 specific choices.

EJS, Server side rendering, ExpressJS: A serverside web application with UI was built using standard technologies and SSR to present analytics data obtained from CW1. Instead of re-engineering CW1, CW2 connects to it via the secured API. EJS was used to make reusable templating, server-side rendering was more secure and dynamic so was chosen.

ChartJS, jspdf, bootstrap: Used to build a responsive and stable UI. ChartJS and jspdf are needed for the dashboard page functionality and bootstrap was used in general for responsiveness across pages.

Objectives Achieved

A secure, horizontal scalable, advanced server-side web application was created using ExpressJS framework. This application provides a user interface and analytics for data collected by the API application developed for coursework 1. Appropriate UI was made, with downloadable and customizable charts and an alumni search feature along with a page to make bids.

Documentation is provided and hosted in the application including API documentation, documentation for the env variables and instructions to get the application up and running both for development and production set-up. Code is kept simple and self-explanatory with comments where necessary.

Testing was created and documented, including the integration test that also help document the code, integration tests that make sure logic works against the API. Additionally, UAT tests were performed for the UI.

Architecture designs are documented in this report and choices have been defended.

Appendix

API Specifics

Authentication

The authentication endpoints allow for creating users (/auth/users/), verifying emails (auth/verify-email), obtaining access tokens by logging in and logging out (auth/sessions), and resetting passwords (auth/password-reset).

/auth/users

Users can be created by sending a POST request to this endpoint; this is not protected by JWT so that users can register accounts freely. On registration, an email is sent to the user with a token to verify the ownership of the registered email; the domain of the email is also restricted allowing only emails with the university domain to be registered. The user cannot log-in without verifying the email address.

/auth/verify-email

The verify email endpoint is used to submit tokens sent to the email via GET request to verify the email, or re-request a verification email by using a post request.

/auth/sessions

You can get and manage JWT tokens with the endpoints here. Sending a DELTE request will log you out and invalidate the tokens. Similarly, a summary of the logins and active tokens can be obtained for debugging and observability purposes. This token is stored in cookies. Logging in will also set the csrf cookie.

/auth/password-resets

These endpoints are used to do password-resets where tokens are sent to the users' emails which need to be used to confirm the password change.

/auth/csrf-token

This endpoint will set the csrf-token, can be used to create accounts ect with csrf protection. Needed because account creation and related endpoints are mutable and so require csrf but do not require logins.

Profile

These endpoints are the ones that alumni would use to create and update their profiles.

/profile

A GET request here would display the alumni’s full profile. A PUT request can be used to update the basic details, with specific endpoints for the other details. This is to avoid request sizes getting too large and preventing accidental edits.

/profile/image

Users can update and delete their profile picture using this endpoint. Images can be uploaded through a multipart/form-data request. The image is intended to be hosted on minio as shared file storage.

/profile/degrees

Used to get, add, edit and delete the degrees a user has completed or enrolled in.

/profile/certifications, /profile/licences, /profile/courses

Used to get, add, edit and delete the credentials a user has completed or enrolled in.

/profile/employment

Used to get, add, edit and delete the employment history of a user.

Sponsorship

These endpoints are used to make and process sponsorships from sponsor orgs and accounts to alumni, so that the alumni can bid for alumni of the day using their sponsorship funds

/sponsorships/organizations

Admins can create Sponsor organizations; Sponsors belong to these organizations and can make sponsorship offers on behalf of them. Admins also assign the sponsor accounts to the sponsor orgs.

/sponsorships/organizations/me

A sponsor user can see what organization they belong to and remove themselves from the org.

/sponsorships/alumni/credentials

A sponsor user can see which alumni have what credentials so they can sponsor the ones they want.

/sponsorships/offers

Sponsors can make offers to credentials alumni have using these endpoints, additionally they can view the offers they have already made and withdraw offers if they have not been accepted yet.

/sponsorships/offers/me

A alumni user can see what offers they have received with endpoint, and accept the ones they want through the /sponsorships/offers/{offerId}/response endpoint.

/sponsorships/balance

An alumni can see how much funds they have available for bidding using this endpoint.

/sponsorships/payouts

Admins can see all the payouts made in the application for Alumni of the day. Alumni can see the payouts they specifically received from the sponsors for winning alumni of the day.

/sponsorships/profit/org

This endpoint can be used by an admin to see the total profit made by the university from the biddings.

Bidding

These endpoints are intended to be used by alumni to place bids for the alumni of the day slot.

/bids

The alumni can place bids for alumni of the day using this endpoint; only increasing the bid is allowed. The status of the bid is displayed as winning or losing.

/bids/summary

This endpoint shows the alumnus their summary for the month, including how many times they have won and how many times they have left to win and the bids they have made for the month.

/bids/current

Returns the details of the alumnus's current active bid. Easier than getting the list of bids and finding the current active bid.

Public API

Used by the 3rd party applications to get the alumni profiles and the alumni of the day.

/public/alumni/featured

Shows the profile of the alumni that won the bid for today. Needs the API client token as authorization.

/public/alumni/{alumniId}

Can be used by the client to view the public profile of a specific alumnus.

/public/alumni/featured/history

This endpoint can be used to see who the previous winners were.

API Client Management

Developer accounts can create and manage clients and their tokens with these endpoints.

/clients

Developer account manages client applications using these endpoints. API keys are created for these clients.

/clients/{clientId}/usage

Shows the usage details of the endpoint access through API key tokens including what tokens are active.

/clients/{clientId}/tokens

Developers can create and revoke API access tokens for each client using this endpoint. The tokens are long lived tokens to be used in the Bearer Header.

Admin

Admin accounts can do management and privileged viewing of data with these endpoints.

/admin/event-attendances

Admins can mark if alumnus attended any event, if they did they are made eligible to win alumni of the day one extra time for that month.

/admin/winners

Admins can manually trigger a winner to be created and list previous winners with this endpoint in case the automatic selection fails.

/admin/users

Used to create a user (only admins can create Sponsor, Developer, Admin accounts). Also can list all the users currently in the application.

/admin/users/{userId}/role

Admins can change the role of users to Alumni, Developer, Sponsor or Admin using this endpoint. Intended to be used as a user management endpoint.

UAT testing

UAT testing was done via postman, the collection is within the repository in alum/.postman.

Test case 001: Create a user

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details (X-CSRF-Token header set)
3. Click link on email received to verify
4. GET 200 response for auth/verify-email/{token} with success message

Test result: Pass

Test Case 002: Create a user with invalid email

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details with email from wrong domain
3. Receive 400 response with error message

Test result: Pass

4. POST /auth/users with user details with email with incorrect formatting
5. Receive 400 response with error message

Test result: Pass

Test Case 003: Create user with same email address

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details with email from wrong domain
3. Receive 400 response with error message

Test result: Pass

Test Case 004: attempt to login without verifying email

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details
3. POST /auth/sessions with user credentials
4. Receive error response

Test result: Pass

Test Case 005: re-request verification email

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details
3. POST auth/verify-email/ with email to get new verification email
4. Click link on new email received to verify
5. GET 200 response for auth/verify-email/{token} with success message

Test result: Pass

Test Case 006: re-request already verified email

1. GET CSRF token from /auth/csrf-token
2. POST /auth/users with user details
3. Click link on email received to verify
4. GET 200 response for auth/verify-email/{token} with success message
5. POST auth/verify-email/ with email to get verification
6. Get error response with message

Test result: Pass

Test case 007: login to receive tokens

Prerequisites: user created

1. GET CSRF token from /auth/csrf-token
2. POST /auth/sessions with user credentials
3. 200 response with success message

4. See that cookie has been set

Test result: Pass

Test case 008: login with invalid credentials

Prerequisites: user created

1. GET CSRF token from /auth/csrf-token
2. POST /auth/sessions with wrong password
3. error response with message
4. POST /auth/sessions with wrong email
5. error response with message

Test result: Pass

Test case 009: get number of active sessions

Prerequisites: user created, user logged in

1. GET CSRF token from /auth/csrf-token
2. POST /auth/sessions/summary
3. Get response with message and correct details

Test result: Pass

Test case 010: logout

Prerequisites: user created, user logged in

1. GET CSRF token from /auth/csrf-token
2. DELETE /auth/sessions
3. POST /auth/sessions/summary
4. Get error response with message with auth error

Test result: Pass

Test case 011: change password

Prerequisites: user created, user logged in

1. GET CSRF token from /auth/csrf-token
2. POST /auth/password-resets
3. Read email for token
4. PATCH /auth/password-resets
5. POST /auth/sessions/summary
6. Get error response with message with auth error
7. Login with new password POST auth/sessions

Test results: Pass

Test case 012: list existing users

Prerequisites: user created, user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. GET /admin/users
3. 200 response with success message

Test results Pass

Test case 013: list existing users with non admin

Prerequisites: user created, user logged in as non-admin

1. GET CSRF token from /auth/csrf-token
2. GET /admin/users
3. Error message
4. Login as sponsor account
5. GET /admin/users
6. Error message

Test results Pass

Test case 014: admin creates a sponsor account

Prerequisites: user created, user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. POST /admin/users with account details
3. 200 response with success message
4. Click link on email received to verify
5. GET 200 response for auth/verify-email/{token} with success message

Test results Pass

Test cases 015: change user to admin

Prerequisites: user created, user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. POST /admin/users/{userID}/role with admin role
3. 200 response with success message
4. Login as new admin account
5. GET /admin/users
6. 200 response with success message

Test results Pass

Test 016: change sponsor (or different) domain email to alumni

Prerequisites: sponsor account created, user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. POST /admin/users/{userID}/role with alumni role
3. Error message

Test results Pass

Test 017: attempt to change last admin to alumnus

Prerequisites: one admin account remaining, user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. POST /admin/users/{userID}/role with alumni role
3. Error message

Test results Pass

Test 018: Sponsor makes sponsorship offer to alumnus's credential without sponsor org

Prerequisites: logged in as sponsor

1. POST /sponsorships/offers with cred details
2. Error message

Test result: Pass

Test 019: Admin creates and edits sponsor org

Prerequisistes: logged in as admin

1. POST /sponsorships/organizations with org details
2. Success message
3. GET /sponsorships/organizations
4. Success message and new org listed
5. GET /sponsorships/organizations/{sponsorOrgId}
6. Success message and correct org details
7. PUT /sponsorships/organizations/{sponsorOrgId} with edited org details
8. GET /sponsorships/organizations/{sponsorOrgId}
9. Success message and correct org details

Test result: Pass

Test 020: Admin assigns sponsor user to sponsor org

Prerequisites: logged in as admin, sponsor role user created

1. PUT /sponsorships/organizations/{sponsorOrgId}/users/{userId}
2. Success message
3. GET /sponsorships/organizations/{sponsorOrgId}/users
4. Success message, user listed correctly
5. PUT /sponsorships/organizations/{sponsorOrgId}/users/{userId} different org

6. Error message

Test result: Pass

Test 021: Admin deletes user from org

Prerequisites: logged in as admin, sponsor org created, sponsor user assigned to org

1. DELETE /sponsorships/organizations/{sponsorOrgId}/users/{userId}
2. Success message
3. GET /sponsorships/organizations/{sponsorOrgId}/users
4. Success message, user not listed

Test result: Pass

Test 022: Admin deletes org while user assigned

Prerequisites: logged in as admin, sponsor org created, sponsor user assigned to org

1. DELETE /sponsorships/organizations/{sponsorOrgId}
2. Error message

Test result: Pass

Test 023: Admin deletes org without users or offers

Prerequisites: logged in as admin, sponsor org created

1. DELETE /sponsorships/organizations/{sponsorOrgId}
2. Success message

Test result: Pass

Test 023: Admin deletes org without users or offers (soft-delete)

Prerequisites: logged in as admin, sponsor org created, offers made from org

1. DELETE /sponsorships/organizations/{sponsorOrgId}
2. Success message

Test result: Pass

Test 024: Sponsor makes offer to alumni

Prerequisites: logged in as sponsor, sponsor org assigned

1. GET /sponsorships/alumni/credentials to see alumni available for sponsorship
2. POST /sponsorships/offers with sponsorship details
3. Success message
4. GET /sponsorships/offers
5. Success message
6. GET /sponsorships/offers/{offerId}

7. Success message

Test result: Pass

Test 025: Sponsor cancels offer to alumni

Prerequisites: logged in as sponsor, sponsor org assigned, offer made to alumni

1. DEL /sponsorships/offers/{offerId}
2. Success message

Test result: Pass

Test 026: Alumnus accepts sponsor offer

Prerequisites: logged in as alumni, sponsor offer made to alumni

1. GET /sponsorships/offers/me
2. PUT /sponsorships/offers/{offerId}/response with accept
3. Success message

Test result: Pass

Test 027: Sponsor attempt to cancel accepted offer

Prerequisites: logged in as sponsor, sponsor offer made to alumni, alumni accepted sponsor offer

1. DEL /sponsorships/offers/{offerId}
2. Error message

Test result: Pass

Test 026: Alumnus rejects sponsor offer

Prerequisites: logged in as alumni, sponsor offer made to alumni

1. GET /sponsorships/offers/me
2. PUT /sponsorships/offers/{offerId}/response with decline
3. Success message

Test result: Pass

Test 027: Alumnus places bid

Prerequisites: logged in as alumni, sponsor offer made to alumni

1. GET /sponsorships/balance to see amount available for bidding
2. POST /bids with bid amount
3. Success message
4. GET /bids has bid listed
5. GET /bids/{bidid} shows bid details
6. GET /bids/current shows the bid details

7. GET /bids/summary shows bid and summary

Test result: Pass

Test 028: Alumnus places bid larger than balance

Prerequisites: logged in as alumni, sponsor offer made to alumni

1. GET /sponsorships/balance to see amount available for bidding
2. PATCH /bids/{bidId} with higher bid amount than available balance
3. Error message

Test result: Pass

Test 029: Alumnus notified of outbid

Prerequisites: logged in as alumni2, alumni1 has winning bid, bigger balance available to alumni2

1. GET /sponsorships/balance to see amount available for bidding
2. POST /bids/ with higher bid amount than alumni1
3. Email sent to alumni1 saying they have been outbid

Test result: Pass

Test 030: Alumnus places losing bid

Prerequisites: logged in as alumni2, alumni1 has winning bid, smaller balance available to alumni2

1. GET /sponsorships/balance to see amount available for bidding
2. POST /bids/ with lower bid amount than alumni1
3. Success message but indicates they are losing

Test result: Pass

Test 031: Alumnus receives payout on winning bid

Prerequisites: user logged in as admin, alumni won the bid

1. GET /sponsorships/payouts endpoint
2. 200 response with success message and payout details

Prerequisites: user logged in as alumnus

1. GET /sponsorships/payouts endpoint
2. 200 response with success message and payout details

Test result: Pass

Test 032: Admin creates developer account

Prerequisites: user logged in as admin

1. GET CSRF token from /auth/csrf-token
2. POST /admin/users with dev account details
3. 200 response with success message
4. Click link on email received to verify
5. GET 200 response for auth/verify-email/{token} with success message

Test result: Pass

Test 033: Developer creates client

Prerequisites: user logged in as developer

1. POST /clients with client details
2. Success message
3. GET /clients lists the created client

Test result: Pass

Test 034: Developer creates public:featured:read token for client

Prerequisites: user logged in as developer, client created

1. POST /clients/{clientId}/tokens with scope and expiry
2. 200 response with success message
3. Token listed in GET /clients/{clientId}/tokens

Test result: Pass

Test 035: Client views alumni of the day with token

Prerequisites: user has client access key, alumni of the day selected

1. GET /public/alumni/featured with Bearer in Authorization Header
2. 200 response with correct alumni profile displayed

Test result: Pass

Test 036: Developer revokes client token

Prerequisites: user logged in as developer, client token created

1. DELETE request at /clients/{clientId}/tokens/{tokenId}
2. 204 response with success message
3. GET /clients/{clientId}/tokens shows the token is revoked

Test result: Pass

Test 037: Client cannot view the alumni of the day with revoked token

Prerequisites: user has client access key, alumni of the day selected, client access key is revoked

1. GET /public/alumni/featured

2. Returns 401 Unauthorized and error message

Test result: Pass

Test 038: Alumnus adds profile picture

Prerequisites: user created, user logged in

1. PUT request /profile/image with image as form-data
2. 200 response with success message
3. Open link in response in browser
4. Successfully view the image

Test result: Pass

Test 039: Alumni deletes profile picture

Prerequisites: user created, user logged in, profile picture updated

1. DELETE request /profile/image
2. 204 response with success message
3. Open old link in response in browser
4. No image at location

Test result: Pass